

iPad CarPlay-style Dashboard — Project Spec

1. Objective

Build a custom infotainment dashboard for a 2019 Honda Amaze (no factory touchscreen/CarPlay). An iPad (WiFi-only, no cellular) is mounted on the dashboard and acts as the display. The goal is a UI that looks and feels as close to real Apple CarPlay as possible — large touch-friendly tiles, dark automotive-style theme, no visible iPadOS chrome (status bar, app switcher gestures, etc.) — not just an iPad running two apps side by side.

The system is personal-use only (single developer, single user, sideloaded via a paid Apple Developer account — no App Store distribution).

Core requirements

- Custom SwiftUI app on the iPad that visually mimics CarPlay (not the real CarPlay framework — that requires OEM head unit hardware this car doesn't have).
- Embedded, fully interactive map (not a mirrored/streamed view).
- Embedded music playback with full Apple Music catalog search (not just library).
- Real-time speedometer + trip computer.
- Reliable GPS, sourced from the iPhone (since the iPad has no GPS chip) via a companion app.
- Internet via the iPhone's Personal Hotspot (already solved, no work needed).
- Auto-launch on car Bluetooth connect (already solved via iOS Shortcuts automation — out of scope for the app itself).

Explicitly rejected approaches (and why)

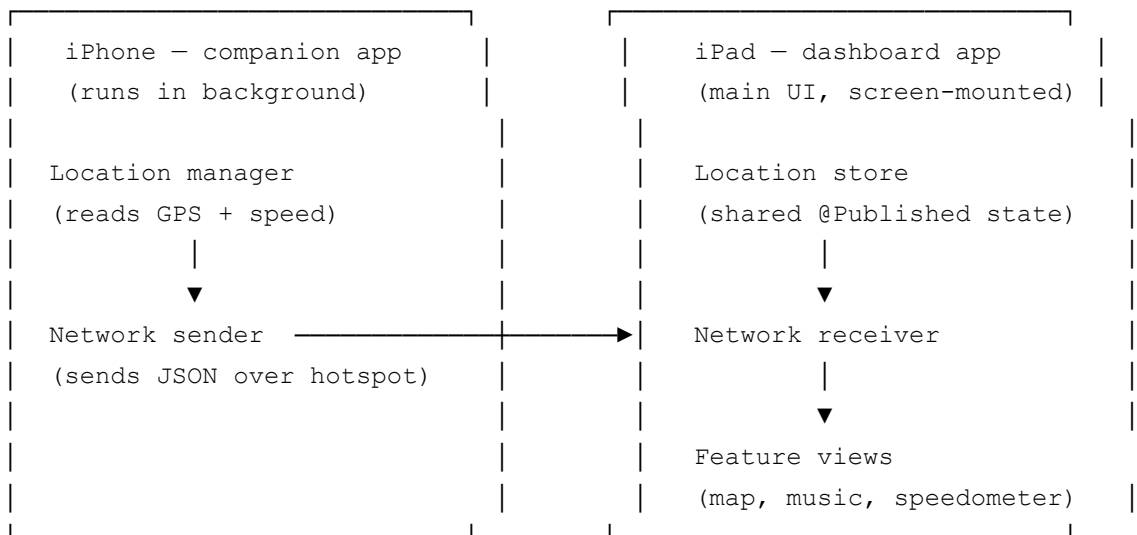
- **Screen-mirroring Google Maps from iPhone to iPad (ReplayKit video stream):** technically possible but rejected. It would be one-way video only — no way to touch-control it from the iPad — and adds encode/network/decode latency that would likely feel worse than the current split-view hack. Use an embedded map SDK instead.
- **Forcing two stock apps into split view programmatically:** not possible via any public iOS API. Solved by building both map and music natively into one app instead.

- **External Bluetooth GPS receiver:** works, but rejected per user preference (no purchase). Solved via companion-app GPS relay instead.

2. High-level architecture

Two separate apps, one on each device, communicating over the existing Personal Hotspot local network. The iPhone acts as network host/server; the iPad connects as client — **discovered via Bonjour, not a hardcoded IP address.**

Design note: An early draft of this spec assumed a fixed/known hotspot IP for the iPhone. That's fine for a single hardcoded personal setup, but it doesn't hold up if this is ever handed to another user with a different phone/network — so the architecture uses Bonjour service discovery from the start instead. This costs almost nothing to build now and avoids a real rework later if the project goes beyond personal use.



Key principle: **one shared source of truth.** The iPad's `LocationStore` is the only thing that receives network data; every feature (map, speedometer, trip computer) reads from it — no feature talks to the network layer directly.

3. Companion app (iPhone) — DashRelay

Purpose: read GPS from the iPhone (which has a real GPS chip, unlike the iPad) and stream it to the iPad continuously, including while backgrounded.

```

├─ DashRelayApp.swift
├─ Info.plist # Background Modes: "Location updates" enable
├─ Services/
│   └─ LocationTracker.swift # Wraps CLLocationManager
│       └─ LocationBroadcaster.swift # NWListener server, advertised via Bonjour;
├─ Models/
│   └─ LocationPacket.swift # Codable: lat, lng, speed, heading, timestam
└─ Views/
    └─ StatusView.swift # Minimal UI: "Relay active" + last-sent time

```

Networking: Bonjour service discovery (not a hardcoded IP)

`LocationBroadcaster` creates an `NWListener` and advertises it as a Bonjour service, e.g.:

```

let listener = try NWListener(using: .tcp)
listener.service = NWListener.Service(name: "DashRelay", type: "_dashrelay._tcp")

```

This means the iPhone announces itself on the local network by name/type rather than by IP — the iPad finds it dynamically no matter what network or hotspot is in use, which is what makes this workable beyond your own single setup.

Info.plist additions required for this:

- `NSLocalNetworkUsageDescription` — a user-facing string explaining why the app needs local network access (iOS shows this as a permission prompt on first launch).
- `NSBonjourServices` — array containing the service type string, e.g. `_dashrelay._tcp`.

Background reliability requirements (critical — read carefully)

1. Request `NSLocationAlwaysAndWhenInUseUsageDescription` authorization (not “While Using”) — required to keep reporting once backgrounded or the phone is locked.
2. Enable the **“Location updates” Background Mode** capability in Xcode (Signing & Capabilities). This is the same mechanism Google Maps/Waze use to stay alive in the background.
3. On `CLLocationManager`: set `allowsBackgroundLocationUpdates = true` and `pausesLocationUpdatesAutomatically = false` (otherwise iOS may auto-pause updates when it thinks you’ve stopped moving, e.g. at a red light).
4. Send the network packet **inside the** `didUpdateLocations` **callback itself** — don’t rely on a separate background timer, which iOS will suspend. Each GPS fix is the wake-up trigger for the network send.

5. iOS will show a persistent blue/green background-location indicator whenever this is active — expected behavior, can't be hidden, and is actually useful as a "is it still running" glance-check.
6. **Never force-quit this app from the App Switcher.** Normal backgrounding (home button, app switch, screen lock) keeps it alive; swiping it away in the App Switcher is treated by iOS as an explicit stop and it will not auto-restart.
7. On the iPad side, implement a watchdog: if no packet has arrived in ~5–10 seconds, show a "GPS signal lost" indicator rather than silently freezing on stale data.

Power

Continuous background GPS + networking will drain the phone — assume it stays plugged into a charger while mounted/used in the car.

4. Dashboard app (iPad) — Dash

Purpose: the actual UI the driver sees and touches. Owns the CarPlay-style visual layer and all embedded features.

```
Dash/
├─ DashApp.swift                # App entry, sets up shared LocationStore
├─ Core/
│   ├─ LocationStore.swift      # ObservableObject, @Published currentLocat
│   ├─ LocationReceiver.swift   # NWBrowser discovers "_dashrelay._tcp", NW
│   └─ ThemeManager.swift       # Dark/light theme switching based on time
├─ Models/
│   ├─ LocationPacket.swift     # Must match companion app's struct exactly
│   └─ TripStats.swift          # distance, duration, avg/max speed for cur
├─ Features/
│   ├─ Map/
│   │   ├─ MapView.swift
│   │   ├─ MapViewModel.swift   # Consumes LocationStore, drives the active
│   │   └─ MapProvider.swift    # Protocol — see Section 5 (Google Maps / A
│   ├─ Music/
│   │   ├─ MusicPlayerView.swift
│   │   └─ MusicPlayerViewModel.swift # MusicKit: catalog search + playback (not
│   ├─ Speedometer/
│   │   ├─ SpeedometerView.swift # Big numeral / gauge, smoothed via short
│   │   └─ TripComputerViewModel.swift # Derives TripStats from LocationStore ov
│   └─ Settings/
│       ├─ SettingsView.swift
│       └─ SettingsStore.swift  # Map provider toggle, persisted via UserD
```

```
└─ Home/
```

```
└─ DashboardView.swift
```

```
# Top-level CarPlay-style tile layout — as
```

Discovering the companion app (iPad side)

`LocationReceiver` uses `NWBrowser` to find the advertised Bonjour service, then opens an `NWConnection` to whichever instance it finds:

```
let browser = NWBrowser(for: .bonjour(type: "_dashrelay._tcp", domain: nil), usin
```

Info.plist additions required (iPad app too):

- `NSLocalNetworkUsageDescription`
- `NSBonjourServices` — must match the same service type string used by the companion app.

Handling more than one discovered instance: on a single personal setup this never comes up, but if this app is ever used by someone with, say, two paired phones nearby, `NWBrowser` could report multiple matches. Don't silently pick one — surface a simple picker ("Connect to: [device name]") the first time, then remember the choice (e.g. by Bonjour service name) so reconnection is automatic afterward.

Reconnection: treat disconnects as expected, not exceptional — the phone could leave Bluetooth range, the app could get backgrounded further than expected, etc.

`LocationReceiver` should re-browse and reconnect automatically rather than requiring the user to relaunch anything, with the existing watchdog (Section 3) covering the gap in the UI while it does.

Shared model code

`LocationPacket` should live in a small local Swift Package imported by **both** Xcode projects, so the wire format can never silently drift between sender and receiver.

5. Map layer — dual-provider design

Build against an abstraction, not a specific SDK, so both Apple Maps and Google Maps can be supported and swapped via a settings toggle.

```
protocol MapProvider {  
    func showLocation(_ coordinate: CLLocationCoordinate2D, heading: Double)  
    func search(query: String) async -> [MapPlace]
```

```

func route(to destination: MapPlace) async -> MapRoute
func draw(route: MapRoute)
}

```

Two concrete implementations: `GoogleMapsProvider` (Google Maps SDK for iOS + Places API + Directions/Routes API — recommended primary for better India road/POI data) and `AppleMapsProvider` (MapKit — free, no API key, good offline-friendly fallback). `SettingsStore` holds which one is active; `MapViewModel` depends only on the `MapProvider` protocol, never on a concrete SDK type directly.

Do not mix data from both providers in the same session (coordinate systems and POI IDs aren't compatible) — one active provider at a time.

Google Maps cost notes (for a single personal user)

- **Maps SDK for iOS (the map view itself): free, unlimited usage.** Can be shown for hours daily at no cost.
- **Places API (search):** free up to 10,000 calls/month — a single user's daily searches are a tiny fraction of this.
- **Directions/Routes API:** billed per route request, not per minute of driving — call it once per trip, not on a timer. ~60–90 trips/month is comfortably within free tier.
- Avoid continuous re-routing timers; only re-call Directions if actually off-route (track position against the cached route locally, which is free).
- Set a Google Cloud billing budget alert (e.g. \$1) as a safety net.

6. Music layer

Use **MusicKit** (`import MusicKit`), not just the local library:

- `MusicAuthorization.request()` for authorization; requires an active Apple Music subscription for full playback.
- `MusicCatalogSearchRequest` — searches the **entire Apple Music catalog**, not just what's already in the user's library. This is the primary search mechanism for `MusicPlayerView`'s search bar.
- Build a fully custom player UI (play/pause/skip/queue/artwork) driven by MusicKit's playback APIs — not by deep-linking into the stock Music app.

7. Speedometer / trip computer

No new data source needed — both are derived entirely from the `LocationPacket` stream already being received:

- **Speed:** `CLLocation.speed` (m/s → convert to km/h), lightly smoothed with a short rolling average (last 2–3 readings) to avoid GPS jitter at low speed.
 - **Heading/compass:** `CLLocation.course` , optional secondary display.
 - **Trip computer:** running distance, elapsed time, average speed, max speed — accumulated in `TripComputerViewModel` , reset on Bluetooth connect (i.e. when a new drive starts).
-

8. Visual design goal (“make it look like CarPlay”)

This is a UI/visual requirement, not just functional:

- Dark, high-contrast theme by default; auto-switch light/dark based on time of day via `ThemeManager` (compute sunrise/sunset from current location).
 - Large, glanceable tiles (map, now-playing, speedometer) — no small text, no dense menus. Design for glances while driving, not close reading.
 - Hide iPadOS chrome as much as possible: run full-screen, disable idle screen dimming while active (`UIApplication.shared.isIdleTimerDisabled = true`), avoid anything that looks like a “regular app” (no navigation bars, no default list styling).
 - A dock-style row of favorite/frequent destinations for one-tap navigation (mirrors real CarPlay’s Maps screen behavior).
 - `DashboardView` (in `Home/`) is where this tile layout and overall visual composition lives — it is the single place that assembles Map + Music + Speedometer into the CarPlay-like grid; individual feature views should not need to know about the overall layout.
-

9. Nice-to-have features (future expansion, not required for v1)

- Weather widget (WeatherKit — free tier)
- Parking-location auto-pin when Bluetooth disconnects (mirrors the existing connect-launch Shortcut)

- Voice control via SiriKit/App Shortcuts or the `Speech` framework for hands-free basics (“play some music”, “navigate home”)
 - Incoming call handling — may already work for free via Continuity/Handoff if both devices share an Apple ID
-

10. Distribution / signing

- Personal use only — no App Store submission needed.
 - A **paid Apple Developer Program account (\$99/year)** is already in place, so provisioning profiles are valid for a full year rather than needing weekly re-signing (the free-tier limitation).
-

11. Build order recommendation

1. `LocationPacket` shared model + companion app’s `LocationTracker / LocationBroadcaster` (background-reliable GPS relay) — everything else depends on this working first.
2. `LocationReceiver` + `LocationStore` on the iPad side, with the watchdog for stale data.
3. `DashboardView` skeleton with placeholder tiles, to validate the CarPlay-style visual layout early.
4. `MapProvider` abstraction + `GoogleMapsProvider` implementation, wired to `LocationStore`.
5. `MusicPlayerViewModel` with `MusicKit` catalog search + playback.
6. `SpeedometerView` + `TripComputerViewModel`.
7. `AppleMapsProvider` as the second `MapProvider` implementation + settings toggle.
8. Polish pass: theming, idle-timer disable, dock/favorites row.